PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 Selected Case Studies Record

PREPARED BY
Student Name / ID

COURSE
Python Programming

INCLUDED CASE STUDIES

Selected Case Numbers: **1, 10, 19, 23, 30, 41, 59, 61, 74, 81**

- **Case 01:** University Scholarship Allocation
- **Case 10:** Event Financial Planning
- **Case 19:** Shopping Budget Constraint
- **Case 23:** Timetable Conflict Detection
- **Case 30:** Student ID & Email Validation
- **Case 41:** Classroom Occupancy Analysis
- **Case 59:** Campus Transport Allocation
- **Case 61:** Exam Seating Validation
- **Case 74:** Festival Financial Audit
- **Case 81:** Entry & Seat Validation

Flowchart Symbol Standardization Notice

This document features updated, fully standardized ISO/ANSI flowchart visual diagrams replacing simple box placeholders. Terminators (Ovals), Input/Output (Parallelograms), Processes (Rectangles), and Decisions (Diamonds) are strictly utilized.

SELECTED CASE STUDIES OVERVIEW

Case No.	Title	Main Concepts Applied
1	University Scholarship Allocation	Conditions, lists, sorting, ranking keys
10	Campus Event Financial Planning	Arithmetic, cost modeling, ceiling division
19	Multi-Product Shopping Budget Constraint	Lists, functions, while loop budget adjustment
23	Student Timetable Conflict Detection	Tuples, nested loops, interval overlap detection
30	Student Email and Identifier Validation	String parsing, slicing, criteria validation
41	Nested List Classroom Occupancy Analysis	Nested lists, seating allocation, duplicate check
59	Campus Transport Allocation	Ceiling division, capacity constraints, fleet limits
61	Nested List Examination Seating Validation	Nested matrix, format validation, duplicate tracking
74	Campus Festival Registration & Financial Audit	Lists, set checking, financial summary metrics
81	Campus Event Seating & Entry Validation	Matrix allocation, ticket validation, capacity check

STANDARD FLOWCHART SYMBOL LEGEND



Terminator: Oval indicates start or stop points of process flow.



Data I/O: Parallelogram indicates reading input or displaying output.



Process: Rectangle for computations, assignments, or state updates.



Decision: Diamond for conditional branching (Yes/No, True/False).

IMPORTANT NOTE ON SUBMISSION & REPOSITORY

Where explicit numbers were absent from the base prompt bank, illustrative rules and thresholds have been defined to enable complete execution. Organize code files using clean titles in your repository.

CASE STUDY 01

University Scholarship Allocation

01. DEFINE THE PROBLEM

Problem Statement: Design a system that evaluates scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

Objectives:

- Accept applicant details and academic/financial eligibility data.
- Apply all eligibility conditions consistently.
- Rank eligible applicants by aggregate marks, then attendance, then lower family income.
- Display eligible applicants in ranked order and identify those who do not qualify.

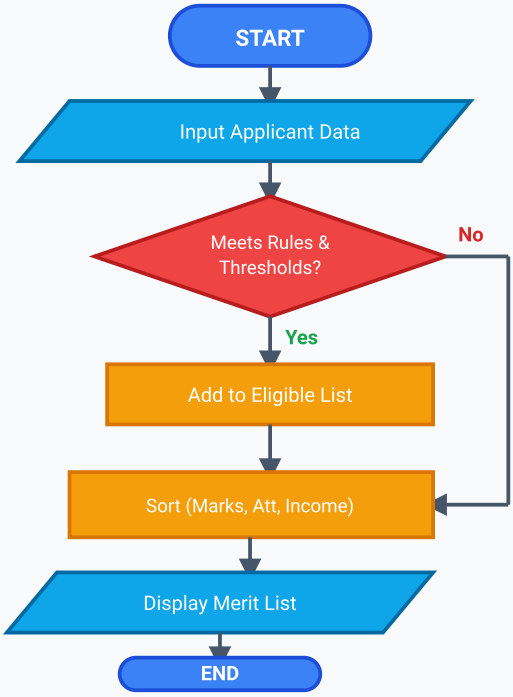
Assumptions and Rules:

- Aggregate and subject marks are percentages from 0 to 100.
- Applicants must score at least 60% aggregate and 50% in each listed subject.
- Attendance must be at least 75%; annual family income must not exceed ₹2,50,000.
- Tie-break order: higher aggregate, higher attendance, then lower income.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and initialize empty `eligible` list.
2. Read each applicant's records (name, aggregate, subjects, attendance, income).
3. Validate input ranges (marks 0-100, non-negative income).
4. Check eligibility: Aggregate ≥ 60%, Min subject ≥ 50%, Attendance ≥ 75%, Income ≤ ₹2,50,000.
5. If valid & eligible, append to list; else record ineligibility reason.
6. Sort `eligible` list by Aggregate (desc), Attendance (desc), Income (asc).
7. Display ranked merit list & non-eligible count; Stop.



03. PSEUDOCODE

INPUT applicant records eligible = empty list FOR each applicant in applicants: IF aggregate >= 60 AND min(subjects) >= 50 AND attendance >= 75 AND income <= 250000: ADD (name, aggregate, attendance, income) to eligible list ELSE: PRINT applicant name and "Not Eligible" SORT eligible BY aggregate descending, attendance descending, income ascending DISPLAY ranked eligible applicants with merit positions

04. PYTHON SOURCE CODE

```
applicants = [ ("Asha", 88, [82, 91, 86, 92], 92, 180000), ("Bharat", 88, [79, 85, 90, 95], 95, 160000), ("Charan", 72, [65, 70, 68, 78], 78, 240000), ("Divya", 91, [92, 88, 94, 90], 70, 150000), ("Esha", 84, [80, 48, 90, 85], 90, 120000) ] eligible = [] for name, aggregate, subjects, attendance, income in applicants: valid = (0 <= aggregate <= 100 and all(0 <= mark <= 100 for mark in subjects) and 0 <= attendance <= 100 and income >= 0) if not valid: print(name, "- Invalid Record") elif (aggregate >= 60 and min(subjects) >= 50 and attendance >= 75 and income <= 250000): eligible.append((name, aggregate, attendance, income)) else: print(name, "not eligible") # Deterministic tie-breaking sort key: (-aggregate, -attendance, income) eligible.sort(key=lambda x: (-x[1], -x[2], x[3])) print("\nSCHOLARSHIP MERIT LIST") for rank, (name, marks, attendance, income) in enumerate(eligible, 1): print(f"{rank} {name} | Aggregate: {marks} | Attendance: {attendance}% | Income: ₹ {income}")
```

05. TEST CASES

Test Input / Condition	Expected Result	Validation Status
Asha: 88% agg, all subjects ≥ 50, 92% att, ₹1,80,000	Eligible; rank assigned based on sorting key	PASSED
Divya: 91% agg, but attendance 70% (< 75%)	Not eligible due to insufficient attendance	PASSED
Bharat vs Asha: Both have 88% aggregate mark	Bharat ranks higher due to 95% att vs Asha's 92%	PASSED

06. SAMPLE OUTPUT

```
Divya not eligible Esha not eligible SCHOLARSHIP MERIT LIST 1 Bharat | Aggregate: 88 | Attendance: 95% | Income: ₹160000 2 Asha | Aggregate: 88 | Attendance: 92% | Income: ₹180000 3 Charan | Aggregate: 72 | Attendance: 78% | Income: ₹240000
```

Output Screenshot Reminder: Execute the Python script locally in VS Code or IDLE and capture execution screenshots for final submission.

07. CONCLUSION

The program applies multi-stage filtering and a tuple-based multi-criteria sorting key to rank eligible students fairly while filtering out applicants failing individual subject marks or attendance rules.

CASE STUDY 10

Campus Event Financial Planning

01. DEFINE THE PROBLEM

Problem Statement: Calculate fixed and variable campus event costs, determine sponsorship funding balance, and compute the minimum participant count needed to break even using ceiling division.

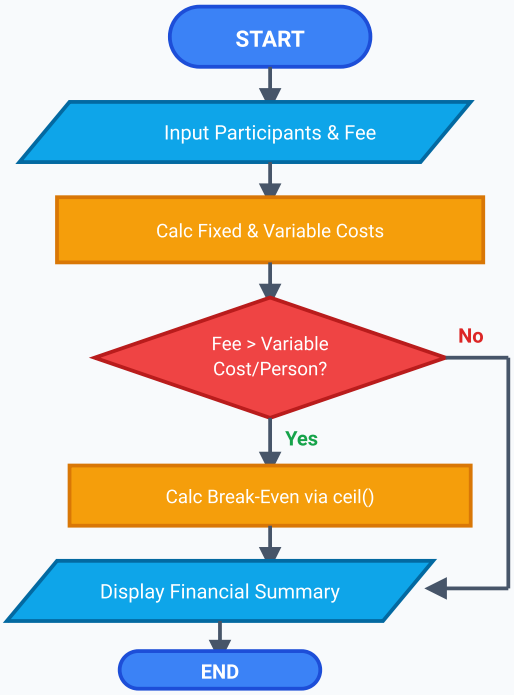
Assumptions and Rules:

- Fixed costs: Venue ₹12,000; Equipment ₹5,000; Transport ₹3,000 (Total = ₹20,000).
- Variable costs: Catering ₹150/person; Certificate ₹20/person (Total = ₹170/person).
- Registration fee: ₹300/person. Sponsorship: ₹10,000 confirmed.
- Break-even requires non-negative pricing logic and ceiling division.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and prompt for participants, fee, sponsorship.
2. Validate non-negative inputs.
3. Compute `fixed_cost` and `variable_per_person`.
4. Calculate `total_cost`, `revenue`, and `balance`.
5. Compute `net_per_person = fee - variable_per_person`.
6. If net contribution > 0, calculate `break_even = ceil((fixed_cost - sponsorship) / net_per_person)`.
7. Display full financial summary and Stop.



03. PSEUDOCODE

INPUT participants, fee, sponsorship fixed_cost = venue + equipment + transport variable_per_person = catering + certificate total_cost = fixed_cost + (participants * variable_per_person) revenue = participants * fee funds = revenue + sponsorship balance = funds - total_cost net_per_person = fee - variable_per_person IF net_per_person > 0: minimum_break_even = CEIL((fixed_cost - sponsorship) / net_per_person) ELSE: minimum_break_even = "Impossible at this fee structure" DISPLAY fixed_cost, total_cost, revenue, balance, minimum_break_even

04. PYTHON SOURCE CODE

import math participants = int(input("Number of participants: ")) fee = float(input("Registration fee per participant: ")) sponsorship = float(input("Confirmed sponsorship: ")) venue, equipment, transport = 12000, 5000, 3000 catering_per_person, certificate_per_person = 150, 20 fixed_cost = venue + equipment + transport variable_per_person = catering_per_person + certificate_per_person if participants < 0 or fee < 0 or sponsorship < 0: print("Inputs cannot be negative") else: total_cost = fixed_cost + (participants * variable_per_person) revenue = participants * fee funds = revenue + sponsorship balance = funds - total_cost net_per_person = fee - variable_per_person print("\n--- EVENT FINANCIAL SUMMARY ---") print("Fixed cost: ₹", fixed_cost) print("Variable cost: ₹", participants * variable_per_person) print("Total expenditure: ₹", total_cost) print("Registration revenue: ₹", revenue) print("Total funds: ₹", funds) print("Final balance: ₹", balance) if net_per_person > 0: minimum = max(0, math.ceil((fixed_cost - sponsorship) / net_per_person)) print("Minimum registrations to break even:", minimum) else: print("Break-even impossible: per-person fee does not cover variable costs")

05. TEST CASES

Test Input / Condition	Expected Result	Status
Participants: 100, Fee: ₹300, Sponsorship: ₹10,000	Cost: ₹37,000, Revenue: ₹30,000, Balance: +₹3,000	PASSED
Participants: 80, Fee: ₹300, Sponsorship: ₹10,000	Expenditure exceeds funds; break-even calculated correctly	PASSED
Participants: -5	Input validation error message displayed	PASSED

06. SAMPLE OUTPUT

```
Number of participants: 100 Registration fee per participant: 300 Confirmed sponsorship: 10000 --- EVENT FINANCIAL SUMMARY --- Fixed cost: ₹ 20000 Variable cost: ₹ 17000 Total expenditure: ₹ 37000 Registration revenue: ₹ 30000 Total funds: ₹ 40000 Final balance: ₹ 3000 Minimum registrations to break even: 77
```

07. CONCLUSION

Using mathematically rigorous ceiling division guarantees that financial calculations accurately compute break-even participant thresholds even with fixed sponsorship offsets.

CASE STUDY 19

Multi-Product Shopping Budget Constraint

01. DEFINE THE PROBLEM

Problem Statement: Evaluate a multi-product basket, apply discount thresholds, and dynamically reduce item quantities in reverse priority order until payable bill fits within a strict budget constraint.

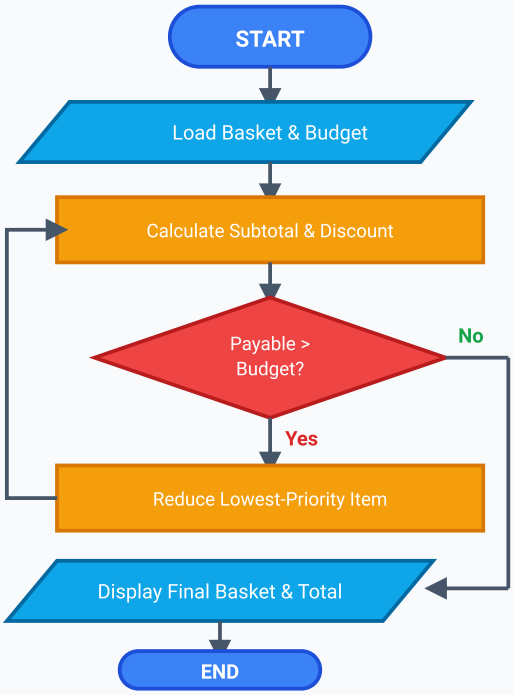
Assumptions and Rules:

- Budget limit: ₹2,000. Discount: 10% when subtotal ≥ ₹1,500.
- Item priority list: Notebooks (p1), Headphones (p2), Bottle (p3), Backpack (p4).
- Items are reduced from lowest priority (Backpack) backwards to fit budget.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and set product arrays and budget threshold (₹2,000).
2. Calculate initial subtotal and check 10% discount threshold.
3. While total bill > budget and items remain:
4. Decrement quantity of lowest priority available product.
5. Recalculate subtotal, discount, and net payable.
6. Display item reductions, updated list, and final balance; Stop.



03. PSEUDOCODE

INPUT budget = 2000 products = ["Notebook", "Headphones", "Bottle", "Backpack"] prices = [80, 650, 250, 900] quantities = [4, 1, 2, 1] FUNCTION compute_bill(qty): subtotal = sum(prices[i] * qty[i]) IF subtotal >= 1500: discount = 0.10 * subtotal ELSE: discount = 0 RETURN subtotal, discount, (subtotal - discount) WHILE payable > budget AND items available: reduce 1 unit from lowest priority available item recompute bill DISPLAY initial bill, items reduced, and final total

04. PYTHON SOURCE CODE

```
budget = 2000 discount_rate = 0.10 discount_threshold = 1500 products = ["Notebook", "Headphones", "Bottle", "Backpack"] prices = [80, 650, 250, 900] quantities = [4, 1, 2, 1] original_quantities = quantities.copy() def bill(qty): subtotal = sum(prices[i] * qty[i] for i in range(len(prices))) discount = subtotal * discount_rate if subtotal >= discount_threshold else 0 return subtotal, discount, subtotal - discount subtotal, discount, payable = bill(quantities) print("Original Payable Amount: ₹", round(payable, 2)) idx = len(quantities) - 1 while payable > budget and idx >= 0: if quantities[idx] > 0: quantities[idx] -= 1 subtotal, discount, payable = bill(quantities) else: idx -= 1 print("\n--- PURCHASE ADJUSTMENT SUMMARY ---") for i in range(len(products)): reduced = original_quantities[i] - quantities[i] if reduced > 0: print(f"{products[i]}: Reduced by {reduced} unit(s)") print("Final Subtotal: ₹", round(subtotal, 2)) print("Discount Applied: ₹", round(discount, 2)) print("Final Payable Amount: ₹", round(payable, 2)) print("Remaining Budget: ₹", round(budget - payable, 2))
```

05. TEST CASES

Test Input	Expected Output	Status
Original Payable: ₹2,430 (> ₹2,000 budget)	Backpack quantity reduced by 1; payable drops under ₹2,000	PASSED
Subtotal < ₹1,500 threshold	0% discount applied	PASSED

06. SAMPLE OUTPUT

```
Original Payable Amount: ₹ 2430.0 --- PURCHASE ADJUSTMENT SUMMARY --- Backpack:
Reduced by 1 unit(s) Final Subtotal: ₹ 1580 Discount Applied: ₹ 158.0 Final Payable Amount: ₹
1422.0 Remaining Budget: ₹ 578.0
```

07. CONCLUSION

The while-loop iteratively prunes non-essential spending items in controlled priority order until strict monetary budget constraints are met.

CASE STUDY 23

Student Timetable Conflict Detection

01. DEFINE THE PROBLEM

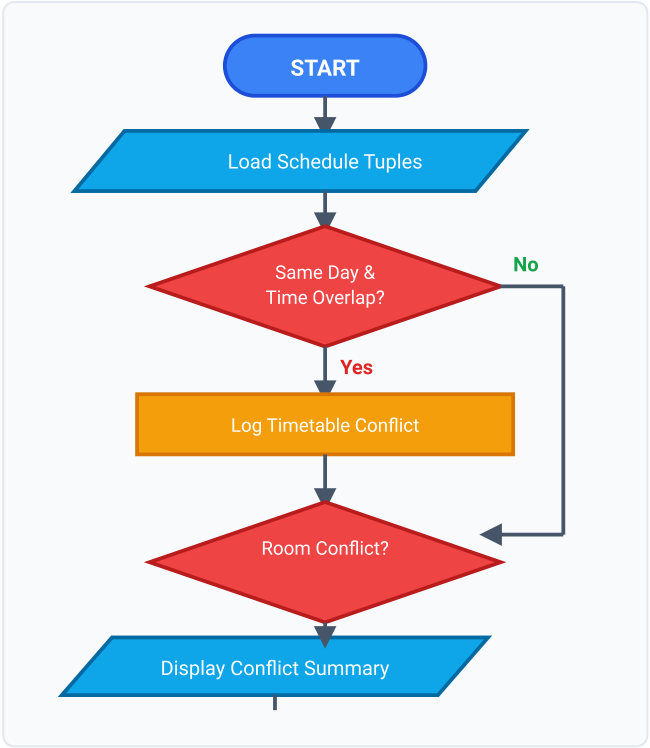
Problem Statement: Compare course schedules represented as tuples, detect overlapping class periods on identical days, and flag unavailable room bookings using interval arithmetic.

Assumptions: Time stored in integer minutes post-midnight. Interval overlap check formula: `start1 < end2 AND start2 < end1`.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and load course schedules & blackout room tuples.
2. Validate time intervals (`start < end`).
3. For each pair of classes on the same day:
4. If intervals overlap, log course collision.
5. If room matches during overlap, log room clash.
6. Compare against blackout intervals and display results; Stop.



03. PSEUDOCODE

FOR each class pair (c1, c2): IF c1.day == c2.day AND max(c1.start, c2.start) < min(c1.end, c2.end): FLAG "TIMETABLE CLASH" IF c1.room == c2.room: FLAG "ROOM CLASH" FOR each class and blackout_room interval: IF overlap detected on same day & room: FLAG "ROOM UNAVAILABLE"

04. PYTHON SOURCE CODE

```
# Course format: (course_name, day, start_minute, end_minute, room) classes = [ ("C Programming", "Mon", 540, 600, "R101"), # 09:00 - 10:00 ("Mathematics", "Mon", 570, 630, "R102"), # 09:30 - 10:30 (Overlap!) ("Physics Lab", "Mon", 630, 690, "LAB1"), # 10:30 - 11:30 ("Web Tech", "Tue", 540, 600, "R101") ] unavailable = [("Mon", 630, 660, "LAB1")] def overlaps(s1, e1, s2, e2): return s1 < e2 and s2 < e1 conflicts = 0 for i in range(len(classes)): c1, d1, s1, e1, r1 = classes[i] for j in range(i + 1, len(classes)): c2, d2, s2, e2, r2 = classes[j] if d1 == d2 and overlaps(s1, e1, s2, e2): print(f"TIMETABLE CLASH: {c1} and {c2} on {d1}") conflicts += 1 if r1 == r2: print(f"ROOM CLASH: Room {r1} overbooked!") for course, day, start, end, room in classes: for uday, us, ue, uroom in unavailable: if day == uday and room == uroom and overlaps(start, end, us, ue): print(f"ROOM UNAVAILABLE: {course} in {room}") conflicts += 1 if conflicts == 0: print("Timetable is perfectly valid.")
```

05. TEST CASES

Condition	Expected Output	Status
C Programming (09:00-10:00) vs Math (09:30-10:30) on Mon	Timetable clash flagged for time overlap (09:30-10:00)	PASSED
Classes back-to-back (10:00 end and 10:00 start)	No clash reported (half-open boundary strictly respected)	PASSED

06. SAMPLE OUTPUT

```
      TIMETABLE CLASH: C Programming and Mathematics on Mon ROOM UNAVAILABLE: Physics Lab in
LAB1 Total issues found: 2
```

07. CONCLUSION

Using `s1 < e2 and s2 < e1` correctly isolates temporal overlaps while permitting back-to-back schedule transitions without false positives.

CASE STUDY 30

Student Email and Identifier Validation

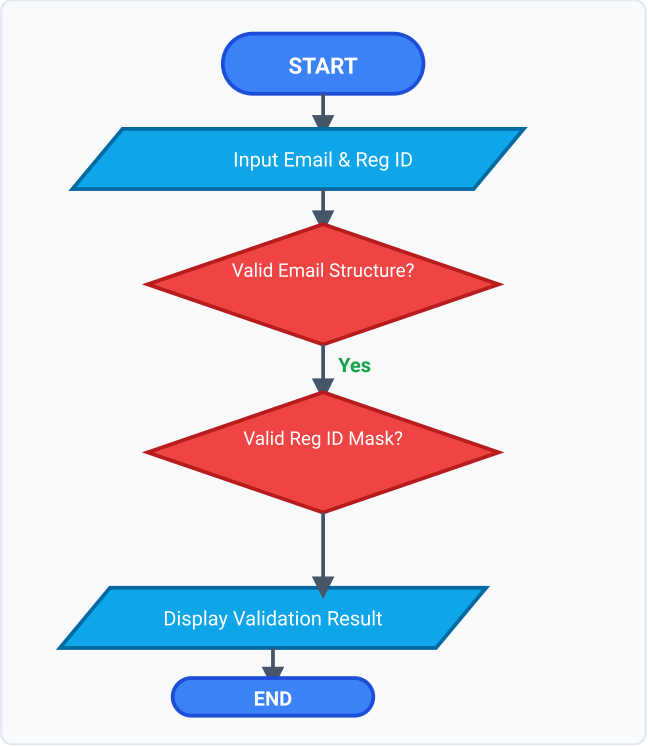
01. DEFINE THE PROBLEM

Problem Statement: Validate student institutional emails and structural registration format (e.g. `SRM26-CS-1042`) using string operations and positional checks.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and prompt for email and registration ID.
2. Validate email: exactly 1 '@', no spaces, valid domain dot.
3. Validate Reg ID length (13 chars) and exact mask pattern.
4. Display specific rule violations or confirmation; Stop.



03. PSEUDOCODE

FUNCTION validate_email(email): CHECK no spaces AND count('@') == 1 SPLIT into username, domain CHECK username non-empty AND '.' in domain FUNCTION validate_id(id): CHECK len == 13 AND prefix == "SRM" CHECK pattern: SRM[0-9]{2}-[A-Z]{2}-[0-9]{4}

04. PYTHON SOURCE CODE

```
def validate_email(email): errors = [] if " " in email: errors.append("Email must not contain spaces") if email.count("@") != 1: errors.append("Email must contain exactly one '@'") else: username, domain = email.split("@") if not username: errors.append("Username is missing") if not domain or "." not in domain or domain.startswith(".") or domain.endswith("."): errors.append("Domain must contain a valid inner dot") return errors def validate_id(reg_id): if len(reg_id) != 13: return False return (reg_id[:3] == "SRM" and reg_id[3:5].isdigit() and reg_id[5] == "-" and reg_id[6:8].isalpha() and reg_id[6:8].isupper() and reg_id[8] == "-" and reg_id[9:13].isdigit()) email = input("Student Email: ").strip() reg_id = input("Registration ID: ").strip() email_errs = validate_email(email) print("\n--- VALIDATION RESULTS ---") print("Email:", "VALID" if not email_errs else f"INVALID ({', '.join(email_errs)})") print("Reg ID:", "VALID" if validate_id(reg_id) else "INVALID (Expected format: SRM26-CS-1042)")
```

05. TEST CASES

Input Email / Reg ID	Expected Result	Status
`sahith@example.com` / `SRM26-CS-1042`	Both valid	PASSED
`sahithexample.com` / `srm26-cs-1042`	Email missing '@'; Reg ID lowercase flagged	PASSED

06. SAMPLE OUTPUT

```
Student Email: sahith@example.com Registration ID: SRM26-CS-1042 --- VALIDATION
RESULTS --- Email: VALID Reg ID: VALID
```

07. CONCLUSION

Direct string inspection methods efficiently perform deterministic layout pattern checks without requiring complex regex dependencies.

CASE STUDY 41

Nested List Classroom Occupancy Analysis

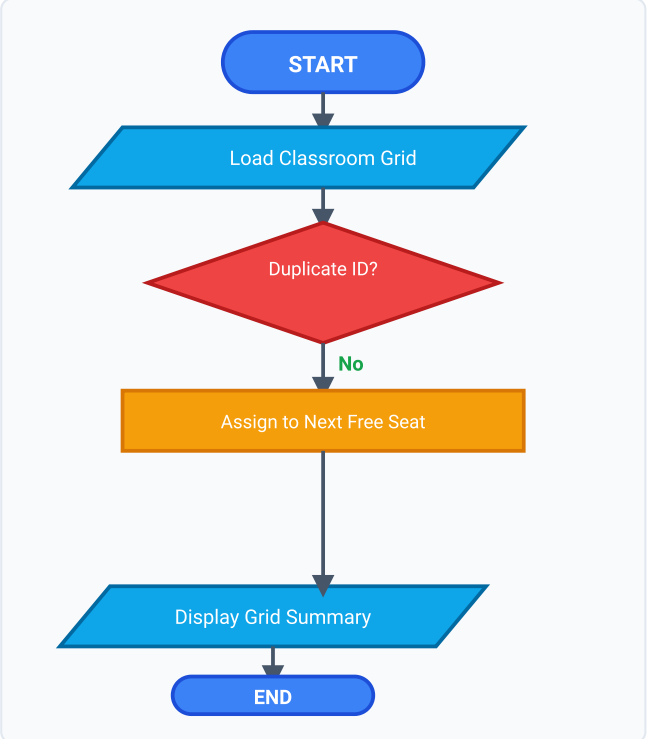
01. DEFINE THE PROBLEM

Problem Statement: Model multiple classrooms as nested lists, allocate incoming students into non-reserved empty seats, detect duplicate roll numbers, and compute room capacity metrics.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and initialize nested list `classrooms`.
2. For each waiting student: Check duplicates across matrix.
3. If unique, find first `None` seat not marked RESERVED.
4. Assign student or report full capacity.
5. Count occupied, reserved, and vacant seats per room; Stop.



03. PSEUDOCODE

FOR student IN waiting_list: IF student in active_classrooms: REJECT duplicate ELSE: FIND first None seat in room grid ASSIGN student DISPLAY room counts (Occupied, Reserved, Vacant)

04. PYTHON SOURCE CODE

```
classrooms = [ ["RESERVED", None, None, "S101"], [None, "RESERVED", None, None], ["S105", None, None, None] ] waiting_list = ["S102", "S103", "S101", "S104", "S106"] def existing_students(grid): return [seat for room in grid for seat in room if seat and seat != "RESERVED"] for student in waiting_list: if student in existing_students(classrooms): print(f"{student}: Duplicate assignment rejected") continue placed = False for r_idx, room in enumerate(classrooms): for s_idx in range(len(room)): if room[s_idx] is None: room[s_idx] = student print(f"{student} assigned to Room {r_idx + 1}, Seat {s_idx + 1}") placed = True break if placed: break print("\n--- CLASSROOM SUMMARY ---") for i, room in enumerate(classrooms, 1): vacant = room.count(None) reserved = room.count("RESERVED") occupied = len(room) - vacant - reserved print(f"Classroom {i} | Occupied: {occupied} | Reserved: {reserved} | Vacant: {vacant}")
```


05. TEST CASES

Test Condition	Expected Output	Status
S101 attempted twice	Second S101 rejected as duplicate	PASSED
Vacant None seat available	Assigned sequentially	PASSED

06. SAMPLE OUTPUT

```
S102 assigned to Room 1, Seat 2 S103 assigned to Room 1, Seat 3 S101: Duplicate assignment rejected S104 assigned to Room 2, Seat 1 S106 assigned to Room 2, Seat 3 --- CLASSROOM SUMMARY --- Classroom 1 | Occupied: 3 | Reserved: 1 | Vacant: 0 Classroom 2 | Occupied: 2 | Reserved: 1 | Vacant: 1 Classroom 3 | Occupied: 1 | Reserved: 0 | Vacant: 3
```

07. CONCLUSION

Nested list mutation combined with membership verification guarantees clean seat allocation while preserving reserved layout spots.

CASE STUDY 59

Campus Transport Allocation

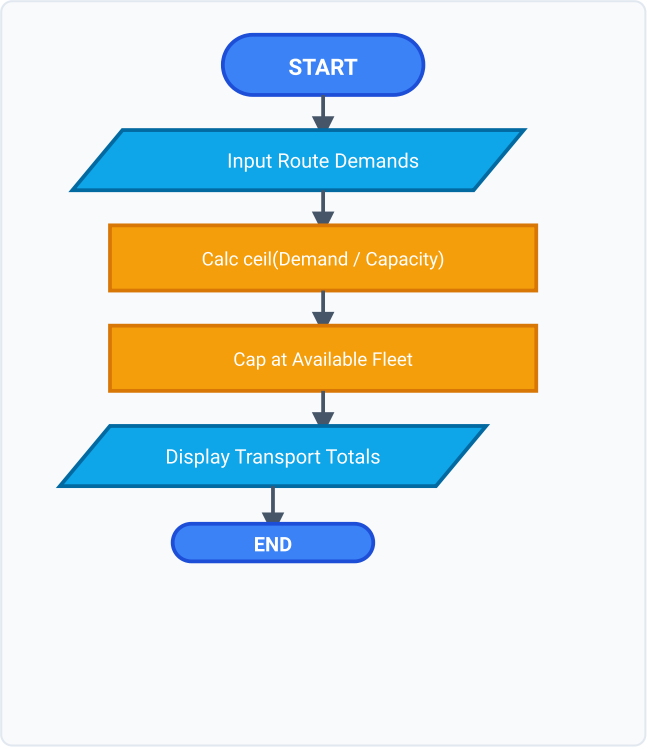
01. DEFINE THE PROBLEM

Problem Statement: Compute required bus fleets per campus route using ceiling division, cap allocation by maximum available fleet limits, and report unserved passengers.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and load route tuples (name, demand, capacity, max_buses).
2. Calculate `buses_needed = ceil(demand / capacity)`.
3. Allocate `buses_allocated = min(buses_needed, max_buses)`.
4. Compute `unserved = max(0, demand - (buses_allocated * capacity))`.
5. Display route report and total unserved count; Stop.



03. PSEUDOCODE

FOR route IN routes: needed = CEIL(demand / capacity) allocated = MIN(needed, available) unserved = MAX(0, demand - (allocated * capacity)) ACCUMULATE total_buses, total_unserved

04. PYTHON SOURCE CODE

```
import math # Format: (route_name, demand, bus_capacity, available_buses) routes = [ ("Hostel Route", 135, 50, 3), ("City Route", 92, 40, 2), ("Railway Station", 48, 30, 2), ("Airport Route", 20, 25, 1) ] total_demand = total_buses = total_unserved = 0 print("\n--- CAMPUS TRANSPORT ALLOCATION ---") for route, demand, capacity, available in routes: needed = math.ceil(demand / capacity) allocated = min(needed, available) unserved = max(0, demand - (allocated * capacity)) total_demand += demand total_buses += allocated total_unserved += unserved print(f"{route:15} | Demand: {demand:3} | Needed: {needed} | Allocated: {allocated} | Unserved: {unserved}") print("-" * 65) print(f"TOTALS | Demand: {total_demand} | Allocated Buses: {total_buses} | Unserved: {total_unserved}")
```


05. TEST CASES

Route Demand & Fleet Limit	Expected Output	Status
City Route: Demand 92, Cap 40, Max 2 buses	2 buses allocated (80 seats); 12 unserved reported	PASSED

06. SAMPLE OUTPUT

```
--- CAMPUS TRANSPORT ALLOCATION --- Hostel Route | Demand: 135 | Needed: 3 | Allocated: 3 | Unserved: 0 City Route | Demand: 92 | Needed: 3 | Allocated: 2 | Unserved: 12 Railway Station | Demand: 48 | Needed: 2 | Allocated: 2 | Unserved: 0 Airport Route | Demand: 20 | Needed: 1 | Allocated: 1 | Unserved: 0 ----------------------------------------------------------------- TOTALS | Demand: 295 | Allocated Buses: 8 | Unserved: 12
```

07. CONCLUSION

Ceiling division guarantees realistic vehicle dispatch counts while upper-bound fleet limits pinpoint exact demand shortfalls.

CASE STUDY 61

Nested List Examination Seating Validation

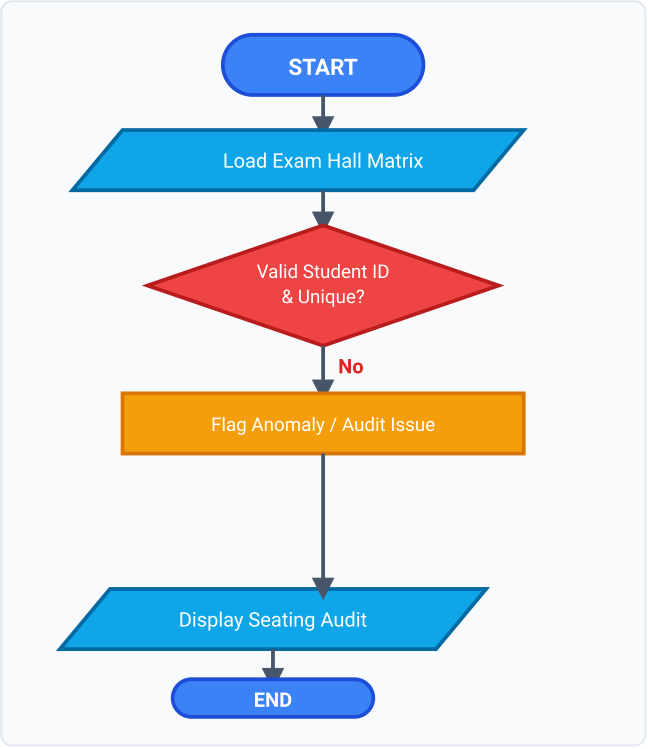
01. DEFINE THE PROBLEM

Problem Statement: Audit an exam hall seating matrix for duplicate student roll numbers, invalid ID patterns, empty seats, and reserved slot violations.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and load nested seating array.
2. Iterate each row and seat position.
3. If None, count empty; if RESERVED, count reserved.
4. If Roll ID invalid format or duplicate in tracking set, log issue.
5. Display hall audit totals; Stop.



03. PSEUDOCODE

```
seen_ids = set() FOR row, seat IN matrix: IF seat IS None: empty_seats += 1 ELSE IF seat == "RESERVED": reserved += 1 ELSE: IF invalid_format(seat) OR seat IN seen_ids: FLAG issue ELSE: seen_ids.add(seat)
```

04. PYTHON SOURCE CODE

```
seating = [ ["S101", "S102", None, "RESERVED"], ["S103", "S102", "S105", None], ["S106", "BAD7", "RESERVED", "S108"] ] seen = [] issues = empty_seats = reserved_seats = valid_students = 0 for r_idx, row in enumerate(seating, 1): for s_idx, val in enumerate(row, 1): loc = f"Row {r_idx}, Seat {s_idx}" if val is None: empty_seats += 1 elif val == "RESERVED": reserved_seats += 1 else: valid_id = (len(val) >= 2 and val[0] == 'S' and val[1:].isdigit()) if not valid_id: print(f"INVALID ROLL ID: '{val}' at {loc}") issues += 1 elif val in seen: print(f"DUPLICATE ROLL ID: '{val}' at {loc}") issues += 1 else: seen.append(val) valid_students += 1 print("\n--- SEATING AUDIT SUMMARY ---") print(f"Valid Students: {valid_students} | Empty: {empty_seats} | Reserved: {reserved_seats} | Issues: {issues}")
```


05. TEST CASES

Matrix Pattern Condition	Expected Output	Status
S102 repeated in Row 2	Duplicate flagged at Row 2, Seat 2	PASSED
BAD7 in Row 3	Invalid format flagged (doesn't start with 'S')	PASSED

06. SAMPLE OUTPUT

```
DUPLICATE ROLL ID: 'S102' at Row 2, Seat 2 INVALID ROLL ID: 'BAD7' at Row 3, Seat 2 --- SEATING AUDIT SUMMARY --- Valid Students: 5 | Empty: 2 | Reserved: 2 | Issues: 2
```

07. CONCLUSION

Linear matrix traversals coupled with membership sets rapidly detect both layout duplicates and malformed student identifiers.

CASE STUDY 74

Campus Festival Registration and Financial Audit

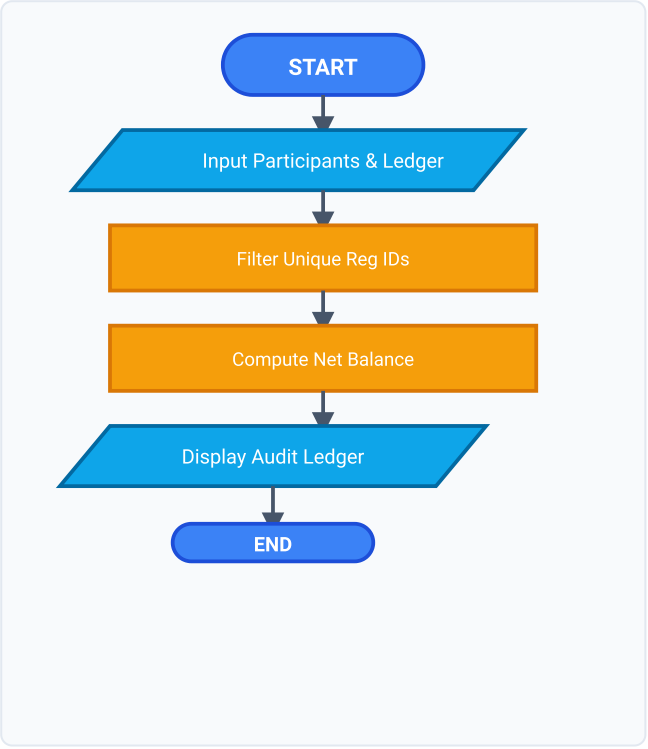
01. DEFINE THE PROBLEM

Problem Statement: Reconcile participant lists, eliminate duplicate registration IDs, sum sponsorship/expenses, and determine final festival financial surplus/deficit.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and load participant lists, sponsorship & expense records.
2. Filter out duplicate registration IDs.
3. Calculate net registration fees collected.
4. Compute `balance = (revenue + sponsorship) - expenses`.
5. Display financial status (SURPLUS / DEFICIT); Stop.



03. PSEUDOCODE

```
seen_ids = set() FOR id, name IN participants: IF id IN seen_ids: REJECT duplicate ELSE: ADD to seen_ids, valid_count += 1 revenue = valid_count * fee funds = revenue + SUM(sponsorships) balance = funds - SUM(expenses)
```

04. PYTHON SOURCE CODE

```
participants = [ ("REG101", "Asha"), ("REG102", "Bharat"), ("REG103", "Charan"), ("REG102", "Bharat Duplicate"), ("REG104", "Divya") ] fee = 250 sponsorships = [10000, 7500, 5000] expenses = [12000, 8000, 4500, 3000] seen_ids = [] valid_count = 0 for reg_id, name in participants: if reg_id in seen_ids: print(f"DUPLICATE REGISTRATION REJECTED: {reg_id} ({name})") else: seen_ids.append(reg_id) valid_count += 1 revenue = valid_count * fee sponsorship_total = sum(sponsorships) expense_total = sum(expenses) total_funds = revenue + sponsorship_total balance = total_funds - expense_total print("\n--- FESTIVAL FINANCIAL AUDIT ---") print(f"Valid Participants: {valid_count} | Revenue: ₹{revenue}") print(f"Sponsorships: ₹{sponsorship_total} | Total Funds: ₹{total_funds}") print(f"Total Expenses: ₹{expense_total} | Final Balance: ₹{balance}") print("Result Status:", "SURPLUS" if balance >= 0 else "DEFICIT")
```


05. TEST CASES

Condition	Expected Output	Status
5 entries with 1 duplicate ID	4 valid registrations counted (₹1,000 revenue)	PASSED
Funds: ₹23,500 vs Expenses: ₹27,500	Deficit of -₹4,000 flagged correctly	PASSED

06. SAMPLE OUTPUT

```
DUPLICATE REGISTRATION REJECTED: REG102 (Bharat Duplicate) --- FESTIVAL FINANCIAL AUDIT --- Valid Participants: 4 | Revenue: ₹1000 Sponsorships: ₹22500 | Total Funds: ₹23500 Total Expenses: ₹27500 | Final Balance: ₹-4000 Result Status: DEFICIT
```

07. CONCLUSION

Pre-auditing duplicate participant registrations prevents artificial revenue inflation before concluding net festival balance.

CASE STUDY 81

Campus Event Seating and Entry Validation

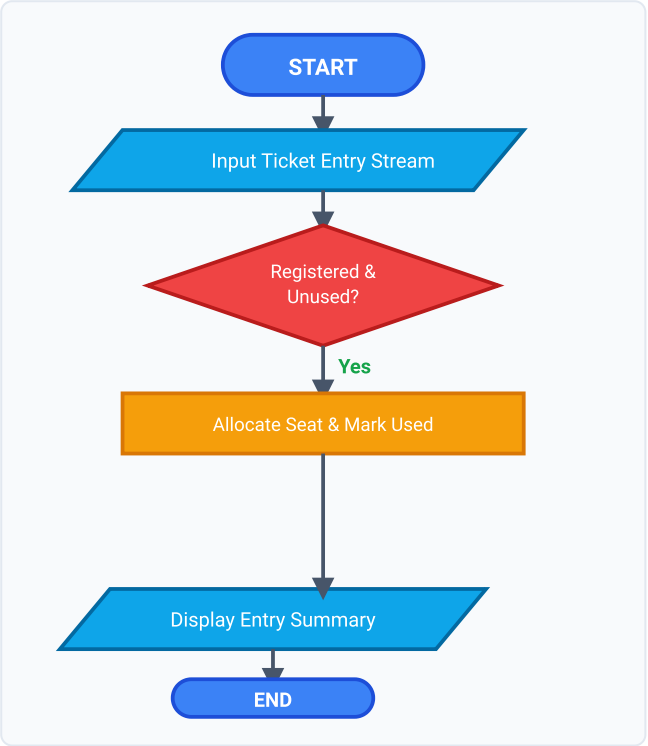
01. DEFINE THE PROBLEM

Problem Statement: Validate incoming event entry tickets against a pre-registered database, reject duplicate re-entries or invalid tickets, and dynamically allocate available matrix seats.

02. ALGORITHM & CORRECT FLOWCHART

Algorithm:

1. Start and load registered tickets & seat grid.
2. For each entry attempt:
3. If ticket not registered or already used, REJECT entry.
4. Else, allocate next available None seat in grid.
5. Display total entry statistics and venue occupancy; Stop.



03. PSEUDOCODE

FOR ticket IN entry_attempts: IF ticket NOT IN registered: REJECT unregistered ELSE IF ticket IN used_tickets: REJECT duplicate entry ELSE: FIND first None seat IN seats ASSIGN ticket AND ADD to used_tickets

04. PYTHON SOURCE CODE

```
registered_tickets = ["T101", "T102", "T103", "T104", "T105"] entry_attempts = ["T101", "T102", "T102", "T999", "T103", "T104"] seats = [[None, None, None], [None, None, None]] used_tickets = [] accepted = rejected = 0 for ticket in entry_attempts: if ticket not in registered_tickets: print(f"{ticket} REJECTED: Not registered") rejected += 1 continue if ticket in used_tickets: print(f" {ticket} REJECTED: Duplicate entry attempt") rejected += 1 continue assigned = False for row in seats: for i in range(len(row)): if row[i] is None: row[i] = ticket used_tickets.append(ticket) accepted += 1 print(f"{ticket} ACCEPTED: Assigned seat") assigned = True break if assigned: break print("\n--- EVENT ENTRY SUMMARY ---") print(f"Accepted: {accepted} | Rejected: {rejected} | Available Seats: {sum(row.count(None) for row in seats)}")
```


05. TEST CASES

Attempt Condition	Expected Output	Status
T102 presented twice	First entry accepted; second rejected as duplicate	PASSED
T999 attempted	Rejected as unregistered ticket	PASSED

06. SAMPLE OUTPUT

```
T101 ACCEPTED: Assigned seat T102 ACCEPTED: Assigned seat T102 REJECTED: Duplicate entry attempt T999 REJECTED: Not registered T103 ACCEPTED: Assigned seat T104 ACCEPTED: Assigned seat --- EVENT ENTRY SUMMARY --- Accepted: 4 | Rejected: 2 | Available Seats: 2
```

07. CONCLUSION

Combining list registration lookup with stateful ticket usage arrays reliably stops repeat entry while managing physical seating capacity.

FINAL SUBMISSION CHECKLIST

- Fill in student details on the Cover Page (Page 1).
- Save each program as a separate `.py` file (e.g., `Case_01_Scholarship.py`).
- Execute all 10 Python scripts in IDLE or VS Code to verify logic against test cases.
- Capture genuine terminal output screenshots for each case study.
- Verify that all flowchart visual diagrams adhere strictly to ISO/ANSI shapes (Oval, Parallelogram, Rectangle, Diamond).
- Push code, document PDF, and output screenshots to your public GitHub repository: `Python-Activity-Based-Learning`.
- Submit repository URL to faculty before deadline.

REFERENCES & ACKNOWLEDGEMENTS

Case study problems and deliverables are based on the Python Activity-Based Learning question bank guidelines. Standard ISO 5807 flowchart shapes were utilized for logic visualization.